

SIMATS ENGINEERING

ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK
Weightage:	30%	Total Marks:	25
CO4 Statement:	<i>Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.</i>		
Student Name:	YAZHINI.E.M	Reg.No.:	192511165
Department:	B.E.CSE	Date:	20/08/2026

ANNEXURE A INDUSTRY PROBLEM SOLVING TASK

- Smartphone Performance Optimization**
A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.
- Cloud Server Memory Bottleneck**
A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.
- Autonomous Vehicle Data Processing**
An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.
- AI Inference Edge Device**
An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.
- High-Performance Gaming Console**
A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

Code of Conduct:

*/ YAZHINI.E.M 192511165 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

Question 1: Smartphone Performance Optimization — A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

Problem Analysis

Lag during app switching occurs primarily because the working set of the newly foregrounded app is not present in the fast on-chip caches, forcing the CPU to fetch instructions and data from slower DRAM or, worse, from swapped/compressed memory. Each memory level in a smartphone SoC trades capacity for speed: L1 is tiny but almost instantaneous, L2 and L3 are larger but slower, and DRAM is large but far slower and more power-hungry relative to on-die cache.

Role of Each Memory Level

Level	Typical Size	Latency	Role in App Switching
L1 (I/D)	32–64 KB per core	~1–4 ns (1–4 cycles)	Holds the immediate instruction stream and hot data of the active thread
L2	256 KB–1 MB per core/cluster	~4–12 ns	Buffers recently evicted L1 lines; reduces stalls during context switches
L3 (SLC)	2–8 MB shared	~15–40 ns	Shared across big/LITTLE clusters; retains data for background apps
DRAM (LPDDR)	6–16 GB	~80–120 ns	Holds full app images, framebuffers; every cache miss pays this penalty

Proposed Hierarchical Redesign

- Increase shared L3/System-Level-Cache size so recently backgrounded apps remain resident, cutting cold-start penalties when the user switches back.
- Use victim/exclusive caching between L2 and L3 to avoid duplicate data and effectively enlarge usable capacity.
- Adopt LPDDR5X/5T DRAM with higher bandwidth and finer-grained power states so idle background apps' memory can be kept in low-power self-refresh instead of being evicted.
- Introduce app-aware cache partitioning (e.g., Intel CAT-style) so the foreground app gets priority allocation in L3, preventing background services from thrashing the cache.
- Use hardware prefetchers tuned to predict app-switch patterns (e.g., pre-warming caches for the most recently used app in the task switcher).

Diagram: Proposed Smartphone Memory Hierarchy

Smartphone SoC Memory Hierarchy Redesign

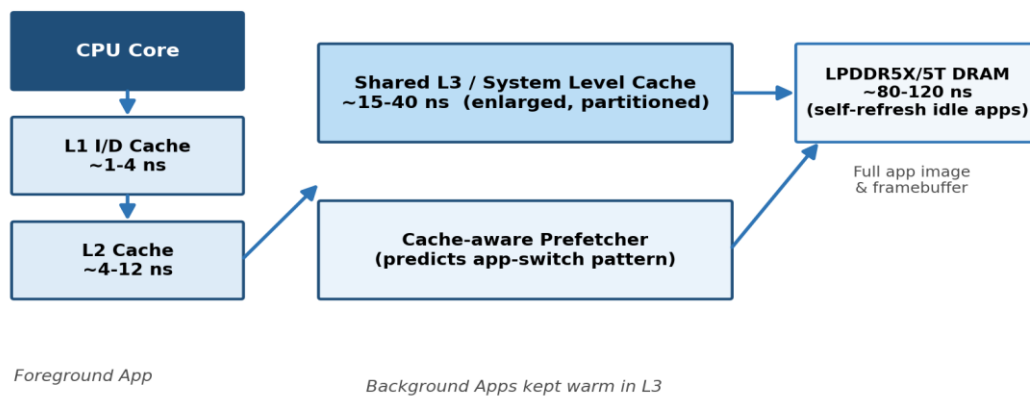


Figure 1: Redesigned SoC cache-to-DRAM hierarchy with L3 partitioning and background-app retention.

Expected Outcome

This redesign reduces the average memory-access time by keeping frequently reused pages closer to the core, reduces DRAM traffic (saving power), and smooths perceived UI latency during multitasking, directly addressing the reported lag.

Question 2: Cloud Server Memory Bottleneck — A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

Problem Analysis

Database workloads on cloud servers are latency-sensitive and memory-intensive. High query latency typically stems from two distinct sources: (1) cache hierarchy misses when the CPU's working set of indexes/hot rows exceeds L1/L2/L3 capacity, and (2) virtual memory paging overhead when the resident set exceeds physical DRAM, causing page faults that go to slower storage (SSD/NVMe).

Cache Hierarchy vs Virtual Memory Paging

Aspect	Cache Hierarchy (L1–L3)	Virtual Memory Paging
Purpose	Speeds up access to frequently used instructions/data	Extends addressable memory beyond physical DRAM
Granularity	Cache lines (64 B)	Pages (4 KB–2 MB, huge pages)
Miss Penalty	Tens of nanoseconds (L3 to DRAM)	Milliseconds (page fault to disk/SSD)
Managed By	Hardware (transparent)	OS + MMU/TLB (software-assisted)
Effect on DB Query	Determines row/index scan speed within RAM	Determines whether data is in RAM at all

Cache misses cost tens of nanoseconds and are largely hidden by out-of-order execution; page faults cost milliseconds and directly stall the query thread. For a database bottleneck, paging overhead is usually the dominant contributor once working-set size approaches physical memory limits, while cache-hierarchy inefficiency dominates when the working set fits in RAM but not in L3.

Recommended Improvements

- Increase L3/last-level cache size and use non-uniform cache access (NUMA)-aware placement so hot index pages stay close to the querying core.
- Enable huge pages (2 MB/1 GB) to reduce TLB misses and page-table walk overhead for large in-memory database buffers.

- Deploy high-bandwidth DDR5/HBM-adjacent memory for the buffer pool to raise throughput for sequential scans and joins.
- Use NVMe SSDs (or persistent memory such as Intel Optane-class technology) as the paging/swap tier instead of SATA SSD or HDD, drastically cutting page-fault latency.
- Right-size physical DRAM (or use memory-tiering software) so the active dataset stays resident, minimizing reliance on paging altogether.

Diagram: Cache Access Path vs Paging Path

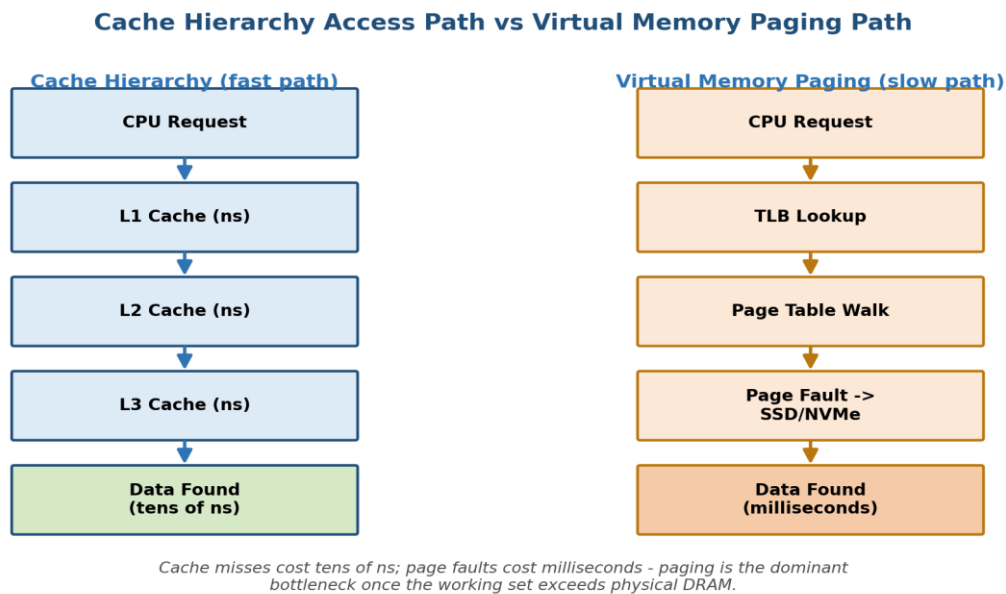


Figure 2: Nanosecond-scale cache path compared with millisecond-scale page-fault path.

Expected Outcome

Combining a tuned cache hierarchy with reduced, faster paging shrinks both nanosecond-scale and millisecond-scale latency components, improving query throughput and tail latency (p99) for the database service.

Question 3: Autonomous Vehicle Data Processing — An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

Problem Analysis

Autonomous vehicles fuse data from LiDAR, radar, and cameras in real time, requiring deterministic, low-latency, high-bandwidth memory for perception and control loops. Any unpredictable stall can delay a safety-critical decision, so the memory hierarchy must balance speed, capacity, persistence, and power.

Evaluation of Memory Technologies

Technology	Latency	Bandwidth	Volatility/Power	Suitability
SRAM	Sub-nanosecond to ~1 ns	Very high (on-chip)	Volatile; higher static power per bit	Ideal for L1/L2 caches and safety-critical control registers
DRAM	~50–100 ns	High (with wide buses/HBM)	Volatile; needs refresh	Best for large frame buffers holding sensor data (point clouds, images)
MRAM	~10–30 ns (STT-MRAM)	Moderate-high	Non-volatile; low standby power	Good for retaining critical state across power glitches/resets

Proposed Hierarchy Design

- L1/L2 SRAM caches tightly coupled to each perception-processing core (DSP/GPU/NPU) for near-instant access to active feature maps.
- On-chip shared L3 SRAM/scratchpad for sensor-fusion intermediate results shared across compute units, avoiding round-trips to DRAM.
- High-bandwidth DRAM (LPDDR5/HBM2E) as main memory for raw sensor buffers (camera frames, LiDAR point clouds) to sustain the multi-GB/s ingest rate.
- MRAM as a non-volatile buffer for critical state (last known safe trajectory, calibration data) so the system can recover instantly after a power interruption without a slow reboot from flash.
- Real-time memory controller with priority-based arbitration ensuring safety-critical control-loop reads always preempt lower-priority logging traffic.

Diagram: Sensor-to-Decision Memory Pipeline

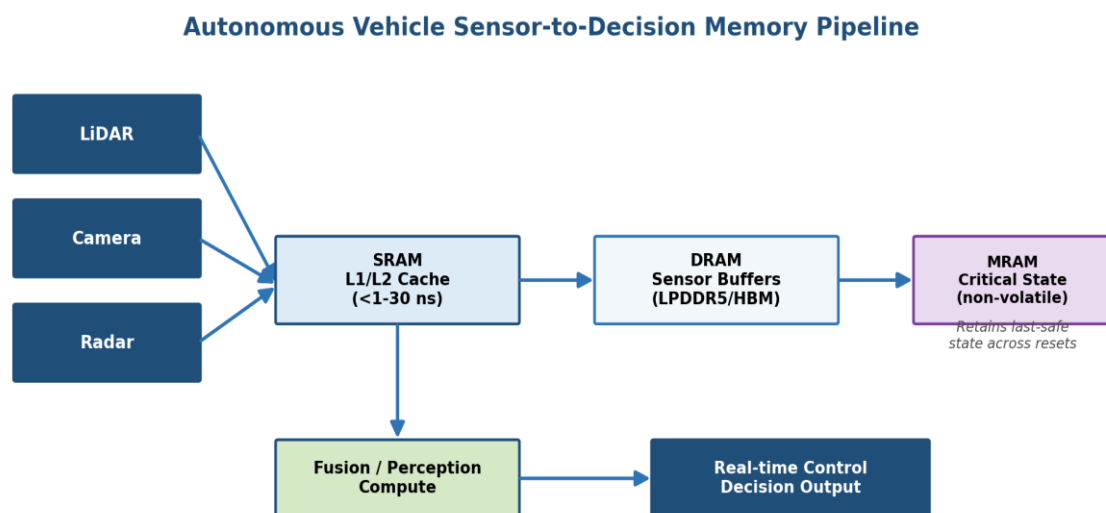


Figure 3: SRAM/DRAM/MRAM tiered pipeline from sensors to real-time control decision.

Expected Outcome

This tiered design gives the perception pipeline the bandwidth needed for continuous multi-sensor streaming (via DRAM), the latency needed for split-second control decisions (via SRAM), and resilience against power events (via MRAM), together minimizing worst-case latency for safety-critical operations.

Question 4: AI Inference Edge Device — An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

Problem Analysis

Edge AI devices (e.g., smart cameras, wearables) are constrained by battery life and must load neural-network weights quickly after wake-up while keeping idle power minimal. The choice of storage/memory technology directly affects both boot/model-load latency and standby power draw.

Comparison of Technologies

Technology	Read Latency	Write/Endurance	Power (Idle)	Role
NAND Flash	Slow (μ s, page-based)	Limited endurance; block erase needed	Very low (non-volatile)	Stores model weights persistently between power cycles
DRAM	Fast (tens of ns)	Unlimited endurance	Needs constant refresh; higher idle power	Working memory for active inference (activations, loaded weights)

Technology	Read Latency	Write/Endurance	Power (Idle)	Role
RRAM	Fast (ns range), improving	Good endurance; non-volatile	Very low; no refresh needed	Candidate for in-memory compute and fast, low-power weight storage

Recommended Hierarchical Structure

- Use RRAM as the primary non-volatile store for pre-trained model weights, since it offers near-DRAM read speed with flash-like persistence and much lower standby power than keeping DRAM alive.
- Reserve NAND Flash for bulk, infrequently accessed data (multiple model versions, logs, firmware) where its higher latency is acceptable.
- Use a modest amount of DRAM strictly as scratch memory for intermediate activations during inference, and place it in self-refresh/power-down mode whenever the device is idle.
- Explore RRAM-based compute-in-memory (CIM) for matrix-multiply-heavy layers to reduce data movement between memory and the inference accelerator, which is often the dominant energy cost in edge AI.
- Cache the most-recently-used model/weights in a small SRAM buffer near the NPU to avoid repeated RRAM reads for weights reused across consecutive inferences.

Diagram: Optimized Edge AI Memory Structure

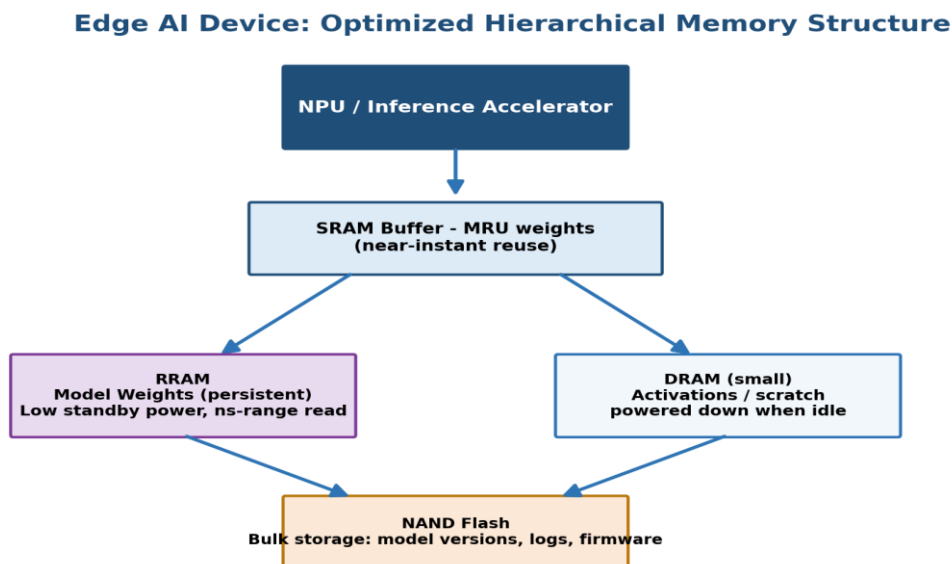


Figure 4: RRAM-resident weights with SRAM reuse buffer and NAND for bulk storage.

Expected Outcome

Replacing a flash-to-DRAM boot path with RRAM-resident weights removes the slow NAND read step from every wake-up cycle, cutting model-load latency, while RRAM's non-volatility allows DRAM to stay powered down between inferences, substantially lowering average power consumption.

Question 5: High-Performance Gaming Console — A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

Problem Analysis

Frame drops during scene loading occur when the GPU/CPU cannot fetch new texture, geometry, and world-state data fast enough to keep the render pipeline fed. This is fundamentally a throughput and streaming-bandwidth problem rather than a pure latency problem, since large, mostly sequential asset loads must move from storage through DRAM and caches into the GPU before each frame deadline.

L3 Cache vs DRAM Throughput

Aspect	L3 Cache	DRAM (e.g., GDDR6/6X)
Bandwidth	Very high but limited to small on-die capacity (a few MB)	High aggregate bandwidth (500+ GB/s) but higher per-access latency
Best For	Reused small working sets: shader state, draw-call metadata	Bulk streaming: textures, meshes, level geometry
Limitation During Load	Too small to hold a new scene's full asset set	Bandwidth can be saturated by simultaneous CPU+GPU+I/O traffic

During large scene loads, L3 cannot help much because the incoming data set vastly exceeds its capacity; the bottleneck shifts to DRAM bandwidth and, further upstream, to storage-to-DRAM transfer speed.

Proposed Memory Hierarchy Enhancements

- Adopt a high-bandwidth unified memory architecture (GDDR6/6X or HBM) shared between CPU and GPU to avoid duplicate copies and maximize effective streaming bandwidth.
- Pair with an NVMe SSD and a hardware decompression block (as in modern consoles) so assets stream directly from storage to DRAM with minimal CPU intervention, reducing load-time stalls.
- Use a dedicated streaming/prefetch cache (separate from the general-purpose L3) that predicts upcoming asset needs based on player position, warming DRAM ahead of the render deadline.
- Increase L3 partition allocated to draw-call and command-buffer data so it is insulated from being evicted by the large asset stream, keeping frame-critical metadata fast even under heavy I/O load.
- Implement double/triple buffering in DRAM for level assets so the next scene is loaded in the background while the current scene continues rendering, hiding load latency behind gameplay.

Diagram: Bandwidth Comparison and Streaming Pipeline

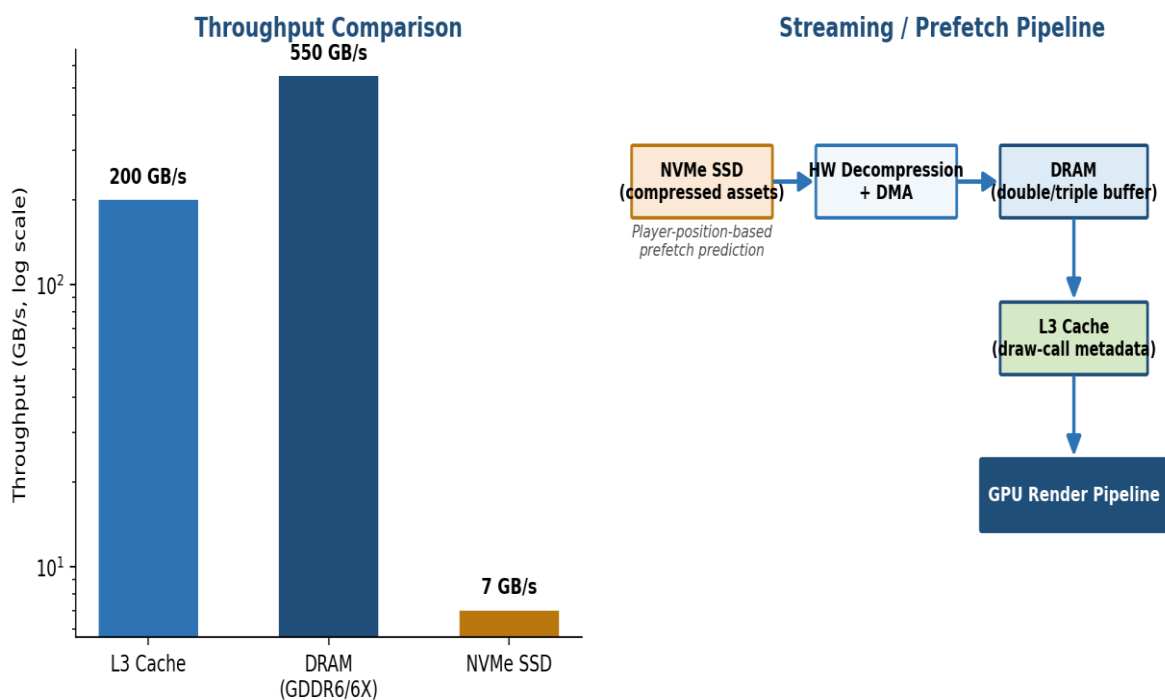


Figure 5: L3/DRAM/NVMe throughput comparison and the asset-streaming/prefetch pipeline.

Expected Outcome

By treating L3 as a small, fast working-set cache and DRAM as a high-throughput streaming tier — backed by fast NVMe storage and predictive prefetching — the console can sustain frame rate through scene transitions instead of stalling on synchronous asset loads.